CSE 211 — Theory of Computation

Deterministic Finite Automata

Lesson notes · Exam approach · TF practice with solutions · Quick notes (Hakim Sir's portion)

How to Read These Notes

EXAM FOCUS	Exam importance, where it appears in the TF collection
MEMORIZE	Rules, tables and formulas to memorize
TRAP	Common mistakes and exam traps
EXAM-READY ANSWER	Model wording you can write in the exam
NOTE	Extra explanation / clarification
Grey box, monospace	Code, worked tables, derivations and hand-drawn sketches
Centered grey box	Formula to remember / final numerical result
Purple heading / ✓ green	Practice questions / their solutions

Contents

Exam Map — What the TF Papers Ask — DFA design is in every single paper
Lesson 3 — The DFA as a Mathematical Object — state, 5-tuple, δ, acceptance, ε / ∅ / {ε}, dead states
Lesson 4 — Designing DFAs: Remember the Right Thing — design recipe, substrings, remainders (mod 3/4/5), constraints
Lesson 5 — Advanced DFAs and Reading DFAs — product construction, sliding window, runs, sum-with-reset, describing a given DFA
Quick Notes — DFA — revision bullets + key diagrams

> **NOTE** Slide numbers refer to the PDF pages of Lec03 - DFA.pdf (42 pages). Every DFA in the solutions was checked by a small program that runs it on all strings up to length 9 (6 for larger alphabets) and compares with the language definition. Practice provenance: Actual = taken from a TF paper; Adapted = TF question with changed details; Constructed = new question in TF style.

Exam Map — What the TF Papers Ask

Session	DFA questions
2023-24	Q1(b) DFA over {a,b}: at most two runs of a's of length exactly three (17) · Q2(b) Show {awa | w ∈ {a,b}∗} is regular with a DFA (15) · Q2(c) Do accepting states affect the rows/columns of a DFA transition table? (5)
2022-23	Q1(b) Sum mod 3 with reset R: can a 2-state DFA (remainder 0 / non-zero) work? (12) · Q1(c) DFA over {a,b} with neither aa nor bb; transition table (13) · Q2(a) Describe the languages of five given finite automata (i–v) (20) · Q3(a) DFA: no run of 0s of even length ≥ 2; design idea (15)
2021-22	Q1(b) DFA (≤ 5 states) over {a,b}: begins with a, no substring aab (10)
2020 (Jan term)	Q2(a) State informally the language of the 5-state DFA (q0…q4) (7)
2019-20	Q1(c) DFA over {0,1,2,R}: running sum, reset on R, accept if sum ≡ 1 or 2 (mod 5) (15) · Q2(a) Role of a state in a finite automaton (5) · Q2(c) DFA for base-3 numbers not divisible by 4 (15)
2017-18	Q2(b) DFAs: binary starting with 1 that is a multiple of 5; a power of 2 (17)
2016-17	Q1(a) Xustaeter (3 colours of stones), 3 stones: state diagram; must-fail, can't-fail, might-fail states (5) · Q1(b) Xustaeter with 4 stones: state diagram and state classes (10) · Q1(c) Symmetries in the diagrams → reduced state-diagram scheme (6) · Q1(d) Reduced diagram for 6 stones; classify states (14) · Q2(a) DFA over {a,b,1,2}: #a divisible by 3 and even digit sum (17)
2015-16	Q1(a) Formally define an FSM; formal description of F1 (Figure 1.a) (8⅓) · Q1(b) What language does F1 recognize? (4) · Q1(c) F2 = F1 plus δ(d,1) = d: language? (4) · Q1(d) Change F1 (same #states, #transitions) so that |L(F3)| = |L(F1)| + 1 — possible? (4) · Q1(e) |L(R1)| = 5 but the DFA accepts 1000 random strings — is the DFA wrong? (3) · Q2(a) DFA for binary numbers NOT divisible by 3, with intuition (8⅓)
2014-15	Q1(a) DFA over {0,1,2,3,R}: sum multiple of 3, reset on R (10) · Q1(b) Alphabet {0,1,2,R}, states remember actual sums — draw; what is the problem? (13⅓) · Q2(b) DFA for binary numbers divisible by 4; explain (13⅓)
2013-14	Q5(c) DFA for {a^p b^q | p + q even} (10⅓)
2012-13	Q7(c) DFA for binary strings whose value is a multiple of 5 (8)
2011-12	Q5(c) DFA for {w | |w| ≤ 5} over {0,1} (5⅓)
2010-11	Q2 DFA over {a,b} with (#a) mod 3 = 1; diagram and formal description (10) · Q4 DFA for strings over {0,1} not containing 110; regular expression (10)
2009-10	Q2 DFA over {a,b}: the first symbol never occurs three times consecutively (abaab, baaabba accepted; abaaab, bbbaa rejected) (10)
2008-09	Q6(a) DFA for binary strings whose value is a multiple of 3 (9)
2007-08	Q1(b) DFA over {a,b}: every a is followed by at least one b; design steps, diagram, table (8) · Q2(c) Most important role of states in an automaton; can it have infinite capacity? (5⅓)
2006-07	Q1(a) DFA: strings starting with 0 of odd length or starting with 1 of even length; diagram + table (10) · Q2(b) DFA for even number of 0s and odd number of 1s; verify with δ̂(q0, 11010) (8)

EXAM FOCUS DFA design appears in all 17 papers — usually two parts per paper, 10–20 marks each. Most repeated, in order: (1) remainder DFAs (binary/base-3 value mod 3, 4, 5; #a mod 3) — 2019-20, 2017-18, 2015-16, 2014-15, 2012-13, 2010-11, 2008-09; (2) running sum with a reset symbol R — 2022-23, 2019-20, 2014-15; (3) substring / pattern constraints (no aa or bb, no aab, no 110, awa) — 2023-24, 2022-23, 2021-22, 2010-11; (4) runs of a symbol — 2023-24, 2022-23; (5) describe the language of a given DFA — 2022-23 (20 marks), Jan-2020, 2015-16. Learn the design recipe in Lesson 4 first, then the remainder trick.

Lesson 3 — The DFA as a Mathematical Object

Slides 1–20 (Lec03 §3A, §3B)

1. What Is a State? (sl. 4–7)

A system reads a sequence of events and cannot remember all of them. A **state is a compressed summary of the past**: it keeps exactly the information needed to decide future behaviour and forgets the rest (sl. 5). A machine can be communicated by a drawing (good for intuition) or by a mathematical specification (good for proofs); a transition table is one convenient way to write the transition function (sl. 6–7).

> **EXAM-READY ANSWER** Role of a state (2019-20 Q2(a), 2007-08 Q2(c)): a state is the automaton's only memory. It classifies all inputs read so far into finitely many groups, so that two inputs in the same group behave identically on every continuation. Capacity: an FA has a **finite** number of states, so it can remember only a bounded amount of information (e.g. a remainder, a parity, the last k symbols). It cannot have infinite capacity — with infinitely many states it would no longer be a finite automaton (it could recognise any language, one state per string, and could not be built). Unbounded memory needs a stack (PDA) or a tape (TM).

2. Formal Definition (sl. 8–9)

$$M = (Q, \Sigma, \delta, q0, F)$$
$$\delta : Q \times \Sigma \to Q$$

- Q — finite set of states; Σ — finite input alphabet;
- δ — transition function: exactly one next state for every (state, symbol);
- q0 ∈ Q — start state; F ⊆ Q — set of accepting (final) states.

3. Running Example: Even Number of 1s (sl. 10–14)

EVEN1 = {w ∈ {0,1}* : w contains an even number of 1s}. Members: ε, 0, 11, 1010; non-members: 1, 10, 111. Remember only the parity: qE (even so far), qO (odd so far).

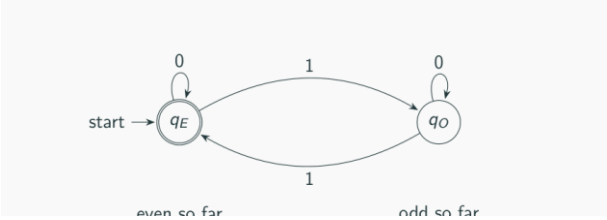


Figure 3.1 — DFA for an even number of 1s (sl. 11)

```
Q = {qE, qO}, Σ = {0,1}, q0 = qE, F = {qE}
      0    1
→*qE  qE   qO
  qO  qO   qE
```

Deterministic (sl. 13): for every (q, a) ∈ Q × Σ there is exactly one δ(q, a) — no guessing, no ambiguity, no missing transition. 'A DFA is finite memory plus a deterministic update rule' (sl. 14).

4. Acceptance and the Language of a DFA (sl. 16–18)

M accepts w = w1w2…wn if there is a sequence of states r0, r1, …, rn with (1) r0 = q0, (2) δ(ri, wi+1) = ri+1 for 0 ≤ i ≤ n − 1, (3) rn ∈ F. L(M) = {w ∈ Σ* : M accepts w}; M recognises L when L(M) = L, i.e. w ∈ L

⇒ accept and w ∉ L ⇒ reject. Extended transition function (textbook): δ̂(q, ε) = q, δ̂(q, xa) = δ(δ̂(q, x), a); M accepts w iff δ̂(q0, w) ∈ F.

> **MEMORIZE** A DFA accepts ε exactly when q0 ∈ F (it reads nothing and stays in q0) (sl. 18).

5. ε, ∅ and {ε}; Dead States (sl. 19–20)

Notation	Meaning	Size
ε	the empty string	length 0
∅	the empty language	0 strings
{ε}	language containing only the empty string	1 string

A **dead state** (trap) is a non-accepting state with all transitions looping to itself. Once entered, success is impossible. Adding one keeps δ total, so the machine stays deterministic and complete (sl. 20). In exam diagrams you may omit it if you say 'missing transitions go to a dead state'.

6. Transition Table Size (2023-24 Q2(c))

The table has one row per state and one column per input symbol: |Q| rows × |Σ| columns. Accepting states are only *marked* (* or a double circle) — changing which or how many states accept never changes the number of rows or columns. (Making more states accepting may let you merge states and so *reduce* |Q|, but that is a different DFA.)

Exam Approach — Formal Description

1. Write the 5-tuple with every component listed explicitly.
2. Give δ as a full table; add a dead state for missing transitions.
3. Mark the start state (→) and accepting states (*).
4. If asked 'what language', trace short strings first, then describe in one sentence.

Practice — Lesson 3

(a) Actual — 2015-16 Q1(a). (b) Actual — 2023-24 Q2(c). (c) Actual — 2006-07 Q2(b) (verification part).

(a) Formally define a Finite State Machine. Formally describe the FSM F1 of Figure 1.a: a −0→ b, b −0→ b, b −1→ c, a −1→ d, d −0→ e; start a; accepting c and e. (8⅓)

(b) Does the number of accepting states in a DFA have any effect on the number of rows and columns in its transition table? Explain. (5)

(c) For the DFA of even 0s and odd 1s (Lesson 5, §1) find δ̂(EE, 11010).

✓ Solution

(a) An FSM (DFA) is a 5-tuple M = (Q, Σ, δ, q0, F) as in §2. For F1:

```
     0   1
→ a  b   d
  b  b   c
 *c  X   X
  d  e   X
 *e  X   X
  X  X   X
(→ start state, * accepting state, X = dead state added for the missing transitions)
```

Q = {a, b, c, d, e} (+ dead state X), Σ = {0, 1}, q0 = a, F = {c, e}. The figure leaves some transitions undefined; formally δ must be total, so all missing moves go to X.

(b) No. Rows = |Q|, columns = |Σ|; accepting states are marked inside the existing rows. See §6.

(c) δ̂(EE, 1) = EO, δ̂(EE, 11) = EE, δ̂(EE, 110) = OE, δ̂(EE, 1101) = OO, δ̂(EE, 11010) = EO ∈ F → accepted (11010 has two 0s and three 1s ✓).

Lesson 4 — Designing DFAs: Remember the Right Thing

Slides 21–29 (Lec03 §3C) + TF design problems

1. The Design Recipe (sl. 22)

1. Decide what information must be remembered — the hardest step: *what finite information is enough?*
2. Make one state for each possible memory situation.
3. Define how each input symbol updates that information.
4. Mark the accepting states. Then add a dead state if needed and test a few members and non-members.

2. Example 1 — Strings Containing 001 (sl. 23–24)

Remember the longest suffix of the input so far that is a prefix of 001: nothing useful (q), 0 (q0), 00 (q00), seen 001 (q001). Note q00 −0→ q00 (the last two symbols are still 00) and q0 −1→ q (a 1 after a single 0 destroys progress).

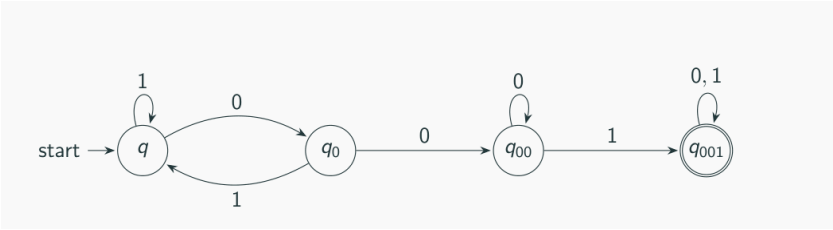


Figure 4.1 — DFA for strings containing 001 (sl. 24)

3. Example 2 — Binary Numbers Divisible by 3 (sl. 25–27)

Appending bit b to a number with value v gives 2v + b. So only the remainder r = v mod 3 matters and **r → (2r + b) mod 3**. A separate start state qs (no bit read) keeps ε out of the language.

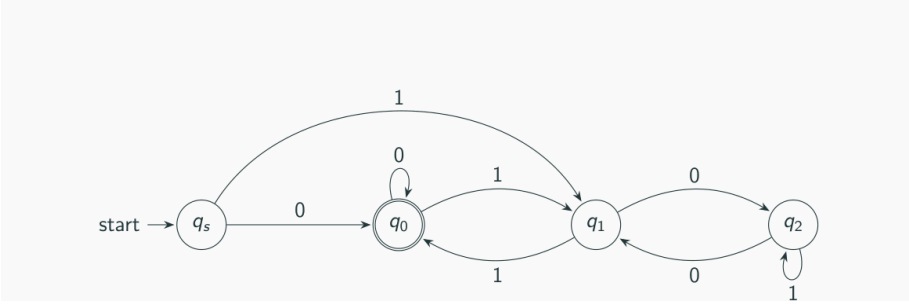


Figure 4.2 — DFA for binary multiples of 3 (sl. 27)

MEMORIZE Remainder DFA for base b and modulus m: states r0 … r(m−1) (+ start qs); δ(r, d) = (b·r + d) mod m; accept the wanted remainders. Works for any base and modulus.

Σ* contains ε, Σ+ = Σ* − {ε} (sl. 25). With the separate start state qs the DFA rejects ε; without it (start = r0) it accepts ε.

4. Example 3 — Valid Football Scores (sl. 28–29)

Σ = {0, d, −} (d = any nonzero digit); L = {x−y : nonnegative integers with no leading zeros}. Remember: first or second score; whether the current score started with 0 (then it must end) or with d; whether the single hyphen is used.

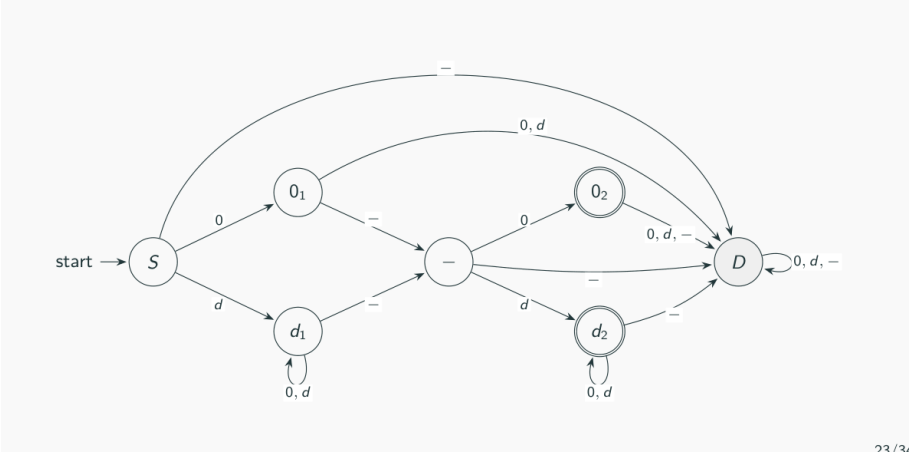


Figure 4.3 — DFA for valid football scores (sl. 29)

Exam Approach — DFA Design

1. Write 2–3 members and non-members (the paper often gives them — test them at the end).
2. Name the memory: remainder, parity, last k symbols, length so far, which pattern progress, 'bad thing seen'.
3. States = values of that memory (+ start state if ε needs special treatment, + dead state).
4. Fill the table row by row — every state needs a move on every symbol.
5. Check: ε, one member, one non-member; then state the meaning of each state in one line (examiners ask for the 'idea').

TRAP For 'divisible' languages decide whether ε is in the language (usually not: add qs). For 'begins with' languages the wrong first symbol goes to a dead state. A 'does not contain' DFA = complement of the 'contains' DFA: flip accepting states of a complete DFA.

Practice — Lesson 4

(a) Actual — 2015-16 Q2(a) / 2008-09 Q6(a). (b) Actual — 2014-15 Q2(b). (c) Actual — 2012-13 Q7(c), 2017-18 Q2(b). (d) Actual — 2019-20 Q2(c). (e) Actual — 2010-11 Q2.

(a) Design a DFA that takes a binary string and recognises numbers NOT divisible by 3 (e.g. rejects 1111 = 15, accepts 111 = 7). Explain your idea. (8⅓) — and the DFA for multiples of 3. (9)

(b) Design a DFA which accepts all and only the binary number strings divisible by 4. Explain. (13⅓)

(c) DFA for binary strings whose value is a multiple of 5; DFAs for strings beginning with 1 that are (i) a multiple of 5 (101, 1010, 1111 in; 0, 100, 0101, 111 out), (ii) a power of 2. (8 + 17)

(d) Design a DFA which accepts the base-3 numbers (digits 0, 1, 2) which are not divisible by 4. (15)

(e) DFA over {a, b} accepting strings with n mod 3 = 1, n = number of a's; transition diagram and mathematical representation. (10)

✓ Solution

(a) Idea: reading bits left to right, the value doubles and adds the bit, so keep r = value mod 3 and use r → (2r + b) mod 3. qs = nothing read. Not divisible ⇒ accept r1, r2; divisible ⇒ accept r0.

```
      0   1
→ qs  r0  r1
  r0  r0  r1
 *r1  r2  r0
 *r2  r1  r2
(→ start state, * accepting state, r = value mod 3; for multiples of 3 make only r0 accepting)
```

Trace 111: qs →1 r1 →1 r0 (3) →1 r1 (7) → accept ✓. Trace 1111: … r1 →1 r0 (15) → reject ✓.

(b) Same idea with m = 4: r → (2r + b) mod 4.

```
      0   1
→ qs  r0  r1
 *r0  r0  r1
  r1  r2  r3
  r2  r0  r1
  r3  r2  r3
(→ start state, * accepting state)
```

Why it works: v mod 4 depends only on the last two bits. Minimising: r1 ≡ r3 (both go to r2 on 0 and to r1/r3 on 1) and qs ≡ r2 (both non-accepting, both go to r0 on 0 and to r1 on 1), so 3 states suffice: {qs, r2} = 'nothing yet or ends in 10', {r1, r3} = 'ends in 1', {r0} = 'ends in 00 (or is 0)'. Example: 1100 (12): qs→r1→r3→r2→r0 ✓.

(c) Multiple of 5: r → (2r + b) mod 5, accept r0:

```
      0   1
→ qs  r0  r1
 *r0  r0  r1
  r1  r2  r3
  r2  r4  r0
  r3  r1  r2
  r4  r3  r4
(→ start state, * accepting state)
```

Beginning with 1: the start state goes to dead on 0, otherwise the same table. Trace 1111: qs→r1→r3→r2→r0 accept (15) ✓; 111: …→r2 reject ✓; 0101 → dead ✓.

```
        0     1
→ qs    dead  r1
 *r0    r0    r1
  r1    r2    r3
  r2    r4    r0
  r3    r1    r2
  r4    r3    r4
  dead  dead  dead
(→ start state, * accepting state, 2017-18 Q2(b)(i))
```

(ii) Powers of 2 written with a leading 1 are exactly 10*:

```
        0     1
→ qs    dead  A
 *A     A     dead
  dead  dead  dead
(→ start state, * accepting state, A = '1 followed by 0s')
```

(d) Base 3: appending digit d gives 3v + d, so r → (3r + d) mod 4. Not divisible ⇒ accept r1, r2, r3.

```
      0   1   2
→ qs  r0  r1  r2
  r0  r0  r1  r2
 *r1  r3  r0  r1
 *r2  r2  r3  r0
 *r3  r1  r2  r3
(→ start state, * accepting state)
```

Check: $11_3 = 4$ → qs→r1→r0 reject ✓; $12_3 = 5$ → qs→r1→r1 accept ✓; $102_3 = 11$ → qs→r1→r3→r3 accept ✓.

(e) Only the number of a's mod 3 matters; b's do not change it.

```
      a   b
→ r0  r1  r0
 *r1  r2  r1
  r2  r0  r2
(→ start state, * accepting state)
```

M = ({r0, r1, r2}, {a, b}, δ, r0, {r1}) with δ as in the table (ri = 'number of a's ≡ i mod 3').

More Lesson 4 Practice — Pattern Constraints

Practice — Lesson 4 (patterns)

(f) Actual — 2022-23 Q1(c). (g) Actual — 2021-22 Q1(b). (h) Actual — 2023-24 Q2(b). (i) Actual — 2010-11 Q4. (j) Actual — 2006-07 Q1(a). (k) Actual — 2007-08 Q1(b). (l) Actual — 2009-10 Q2. (m) Actual — 2011-12 Q5(c).

(f) Construct a DFA for {w ∈ {a,b}* | w has neither aa nor bb as a substring}; give the transition table. (13)

(g) DFA with not more than five states over {a, b} accepting strings that begin with a but do not contain aab. (10)

(h) Show that L = {awa | w ∈ {a,b}*} is regular by constructing a DFA for it. (15)

(i) DFA for strings over {0,1} not containing 110; regular expression for the language. (7 + 3)

(j) DFA for strings that start with 0 and have odd length, or start with 1 and have even length; diagram and table. (10)

(k) DFA over {a, b} in which each a is followed by at least one b; design steps and table. (8)

(l) DFA over {a, b} accepting strings in which the symbol that starts the string never occurs three times consecutively (abaab, baaabba accepted; abaaab, bbbaa rejected). (10)

(m) DFA for {w | |w| ≤ 5} over {0, 1}. (5⅓)

✓ Solution

(f) Remember the last symbol (and whether we have started). Seeing it again → dead.

```
     a   b
→*S  A   B
 *A  D   B
 *B  A   D
  D  D   D
(→ start state, * accepting state, A = last symbol a, B = last symbol b, D = dead)
```

L = alternating strings: ε, a, b, ab, ba, aba, … Trace abab: S→A→B→A→B accept; abba: …→B→D reject ✓.

(g) States: S (nothing read), A (last symbol a, previous not a), AA (last two aa), B (last symbol b), D (dead). Wrong first symbol b → D; aa followed by b → D. Exactly 5 states.

```
      a   b
→ S   A   D
 *A   AA  B
 *AA  AA  D
 *B   A   B
  D   D   D
(→ start state, * accepting state)
```

Trace abaa: S→A→B→A→AA accept ✓; aab: S→A→AA→D reject ✓; ba → D ✓.

(h) Need: first symbol a, length ≥ 2, last symbol a. After the first a, remember whether the last symbol read was a (Q, accepting) or b (P).

```
     a   b
→ S  P   D
  P  Q   P
 *Q  Q   P
  D  D   D
(→ start state, * accepting state)
```

aa: S→P→Q accept (w = ε) ✓; a: S→P reject ✓; abba: S→P→P→P→Q accept ✓; b… → D. Since a DFA exists, L is regular.

(i) Track the suffix: q (no progress), q1 (last 1), q11 (last 11); q11 −0→ D. Once 11 has appeared, only 1s can follow.

```
       0    1
→*q    q    q1
 *q1   q    q11
 *q11  D    q11
  D    D    D
(→ start state, * accepting state)
```

RE: (0 ∪ 10)∗1∗

Before the first 11 the string is built from blocks 0 and 10; after the first 11 only 1s may follow (checked by program against all strings up to length 12).

(j) Remember the first symbol and the parity of the length.

```
         0      1
→ s      0odd   1odd
 *0odd   0even  0even
  0even  0odd   0odd
  1odd   1even  1even
 *1even  1odd   1odd
(→ start state, * accepting state)
```

(k) Interpret 'followed by' as 'immediately followed by': L = (b ∪ ab)*. q1 = an a is waiting for its b.

```
      a   b
→*q0  q1  q0
  q1  d   q0
  d   d   d
(→ start state, * accepting state)
```

> **NOTE** If the examiner means 'followed somewhere later by a b', the condition only fails when the string ends in a: then the DFA has q0 (last symbol b or ε, accepting) and q1 (last symbol a, non-accepting) with q0 −a→ q1, q1 −a→ q1, both −b→ q0. State your interpretation.

(l) After the first symbol x, count the current run of x's (0, 1, 2); a third x in a row → dead. The other symbol resets the count.

```
      a   b
→*s   A1  B1
 *A0  A1  A0
 *A1  A2  A0
 *A2  D   A0
 *B0  B0  B1
 *B1  B0  B2
 *B2  B0  D
  D   D   D
(→ start state, * accepting state, Ai = first symbol a, current run of a's = i; Bi likewise for
b)
```

abaab: s→A1→A0→A1→A2→A0 accept ✓; abaaab: …→A2 −a→ D reject ✓; bbbaa: s→B1→B2→D reject ✓.

(m) Count the length up to 6: states q0 … q5 accepting, q6 = dead (all lengths ≥ 6).

```
      0   1
→*q0  q1  q1
 *q1  q2  q2
 *q2  q3  q3
 *q3  q4  q4
 *q4  q5  q5
 *q5  q6  q6
  q6  q6  q6
(→ start state, * accepting state)
```

Lesson 5 — Advanced DFAs and Reading DFAs

Slides 30–42 (Lec03 §3D) + TF problems

1. Product Construction: Two Conditions at Once (sl. 31–32)

L = {w : even number of 0s AND length a multiple of 3}. Track two independent things: parity of 0s (2 values) and length mod 3 (3 values) → 2 × 3 = 6 states (p, l). Accept only (E, 0).

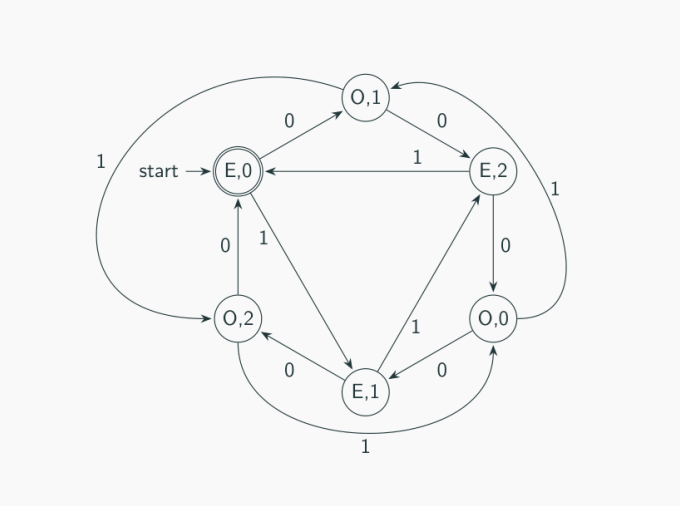


Figure 5.1 — Product DFA: parity of 0s × length mod 3 (sl. 32)

MEMORIZE Product construction: states Q1 × Q2, start (q1, q2), δ((r, s), a) = (δ1(r, a), δ2(s, a)). AND → accept F1 × F2; OR → accept pairs with r ∈ F1 or s ∈ F2. Number of states = |Q1|·|Q2|.

2. The k-th Symbol from the End (sl. 33–34)

L = {w : the 2nd symbol from the right is 1}. Remember the last 2 symbols (sliding window): q00, q01, q10, q11; start q00 (pretend leading zeros); accept q10, q11. From q01 on 0 the window becomes 10.

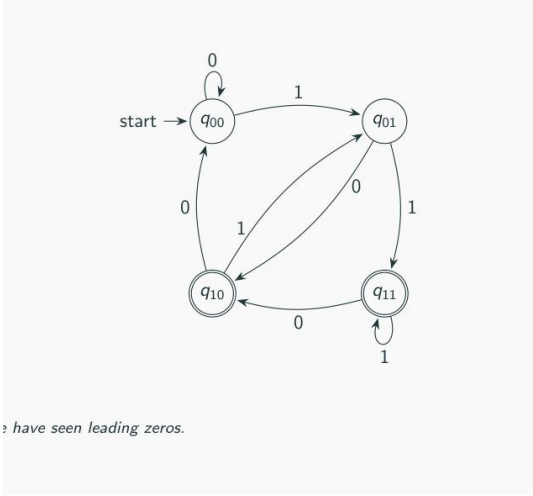


Figure 5.2 — Sliding-window DFA (sl. 34)

In general the k-th symbol from the end needs 2^k states — a DFA cannot 'guess' where the end is (an NFA can, with k + 1 states — Lesson 9).

3. Pattern Interference, Sets of Seen Symbols, Comments (sl. 35–40)

- Contains ab but not ba over {a, b} = a+b+: states S, A, B, AB, Dead (sl. 35–36).

- No symbol repeated over {a, b, c}: state = set of symbols already seen (∅, a, b, c, ab, ac, bc, abc) + D; all except D accept (sl. 37–38).
- Exactly one C comment /…/ over {/, , *x*}: *q0* −/→ *q1* −→ q2 (inside), q2 −*→ q3, q3 −*→ q3, q3 −x→ q2, q3 −/→ q4 (accept); everything else → d (sl. 39–40).

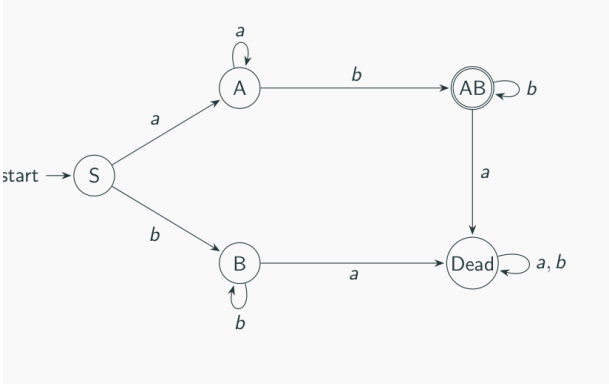


Figure 5.3 — Contains ab but not ba (sl. 36)

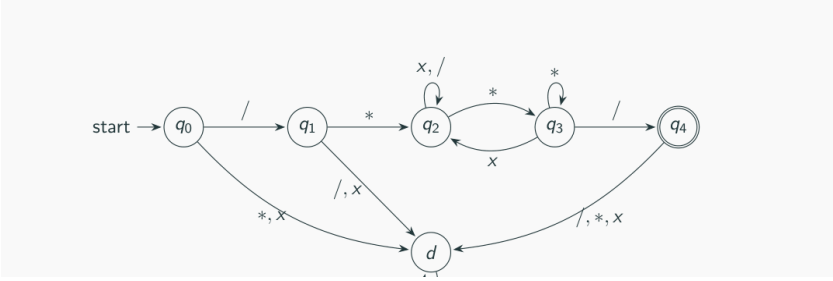


Figure 5.4 — DFA for one C-style comment (sl. 40)

4. Running Sum with Reset (TF favourite)

Σ = {0, 1, 2, …, R}: keep only the sum mod m; a digit d moves r → (r + d) mod m; R moves every state to r0. The actual sum is unbounded, so 'remember the actual sum' needs infinitely many states — not a finite automaton.

5. Reading a Given DFA

1. Find the dead states and the accepting states.
2. Give each state a meaning ('last symbol was a', 'one a pending', 'counter = 2').
3. Trace the shortest accepted strings; look for the loops that pump them.
4. Write the language in English and, if easy, as a regular expression. Check one non-member.

Exam Approach — Two Conditions / Counting

1. Split the condition into independent parts; DFA for each.
2. Combine by the product construction (states = pairs); AND/OR decides the accepting pairs.
3. For runs, keep (number of completed qualifying runs, length of the current run) — cap counters at the largest value that matters.

Practice — Lesson 5

(a) Actual — 2019-20 Q1(c). (b) Actual — 2014-15 Q1(a)–(b). (c) Actual — 2022-23 Q1(b). (d) Actual — 2016-17 Q2(a). (e) Actual — 2006-07 Q2(b). (f) Actual — 2013-14 Q5(c).

(a) DFA with alphabet {0, 1, 2, R} that keeps a running count of the sum of the numerical symbols, resets to 0 on R, and accepts if the sum is 1 or 2 modulo 5. State diagram only. (15)

(b) DFA over {0, 1, 2, 3, R}: sum a multiple of 3, reset on R. Then with alphabet {0, 1, 2, R} you use the states to remember the actual sums — show how; what is the problem? (10 + 13⅓)

(c) For the sum-mod-3 automaton with reset, a classmate claims two states suffice: 'remainder 0' and 'remainder non-zero'. Comment analytically. (12)

(d) DFA over {a, b, 1, 2} accepting strings whose number of a's is divisible by 3 and whose digit sum is even (ε, aab12a1, 121 accepted; aaab12a1, 1121 rejected). (17)

(e) DFA for strings with an even number of 0s and an odd number of 1s. (8)

(f) DFA for L = {w ∈ {a,b}* | w = a^p b^q, p + q even}. (10⅓)

✓ Solution

(a) State ri = (sum since the last R) mod 5. Digit d: ri → r(i+d mod 5); R: every state → r0. Accept r1, r2.

```
      0   1   2   R
→ r0  r0  r1  r2  r0
 *r1  r1  r2  r3  r0
 *r2  r2  r3  r4  r0
  r3  r3  r4  r0  r0
  r4  r4  r0  r1  r0
(→ start state, * accepting state)
```

Trace 2R12: r0→r2→r0→r1→r3 → reject (sum 3) ✓; 2R2R1 → r1 accept ✓.

(b) Same with m = 3 and digits 0–3, accept r0:

```
      0   1   2   3   R
→*r0  r0  r1  r2  r0  r0
  r1  r1  r2  r0  r1  r0
  r2  r2  r0  r1  r2  r0
(→ start state, * accepting state)
```

Remembering actual sums: states s0, s1, s2, … with si −d→ s(i+d), si −R→ s0, accepting s0, s3, s6, …. The sum can grow without bound, so the diagram needs **infinitely many states** — it is not a finite automaton and cannot be drawn completely. Only the remainder mod 3 is needed for the decision, so states {s0, s3, s6, …} can be merged into r0, etc., giving the 3-state DFA.

(c) The claim is false. Use the inputs 1 and 2: both would go to 'non-zero'. Append 2: 12 has sum 3 (accept) but 22 has sum 4 (reject). A DFA in the same state after 1 and after 2 must give the same answer on both continuations — contradiction. Formally, the remainders 0, 1, 2 are pairwise distinguishable (0 vs 1 by suffix ε, 1 vs 2 by suffix 2, 0 vs 2 by ε), so any DFA needs at least 3 states (Myhill–Nerode). The 2-state machine also cannot define its next state: from 'non-zero' on input 1 the result is non-zero for remainder 1 but zero for remainder 2.

```
      0   1   2   R
→*r0  r0  r1  r2  r0
  r1  r1  r2  r0  r0
  r2  r2  r0  r1  r0
(→ start state, * accepting state, the correct 3-state DFA)
```

(d) Product of 'a's mod 3' (0, 1, 2) and 'digit-sum parity' (E, O): 6 states kE/kO. a: k → k+1 mod 3; b: no change; 1: flip parity; 2: keep parity. Accept 0E only.

```
      a   b   1   2
→*0E  1E  0E  0O  0E
  0O  1O  0O  0E  0O
  1E  2E  1E  1O  1E
  1O  2O  1O  1E  1O
  2E  0E  2E  2O  2E
  2O  0O  2O  2E  2O
(→ start state, * accepting state)
```

aab12a1: 0E→1E→2E→2E→2O→2O→0O→0E accept ✓. 1121: 0E→0O→0E→0E→0O reject ✓.

(e) States = (parity of 0s, parity of 1s); start EE; accept EO.

```
      0   1
→ EE  OE  EO
 *EO  OO  EE
  OE  EE  OO
  OO  EO  OE
(→ start state, * accepting state)
```

(f) Remember: still in the a-block or already in the b-block, and the parity of the length; an a after a b → dead.

```
      a   b
→*Ea  Oa  Ob
  Oa  Ea  Eb
 *Eb  D   Ob
  Ob  D   Eb
  D   D   D
(→ start state, * accepting state)
```

ε, aa, ab, abbb accepted; a, aab rejected; ba → D ✓.

More Lesson 5 Practice — Runs and Reading DFAs

Practice — Lesson 5 (runs, reading)

(g) Actual — 2022-23 Q3(a). (h) Actual — 2023-24 Q1(b). (i) Actual — 2022-23 Q2(a). (j) Actual — Jan-2020 Q2(a). (k) Actual — 2015-16 Q1(b)–(e).

(g) DFA accepting binary strings such that none of their runs of 0s has an even length ≥ 2 (0001100 rejected; 01110001, 11111 accepted); give your design idea. (15)

(h) A run is a maximal substring of length ≥ 2 of one symbol. DFA over {a, b} for strings with at most two runs of a's of length exactly three. (17)

(i) Describe informally the languages accepted by the five automata of 2022-23 Q2(a). (20)

(j) State informally the language accepted by the 5-state DFA of Jan-2020 Q2(a). (7)

(k) What language does F1 recognise? F2 = F1 + δ(d, 1) = d? Can F1 be changed (same numbers of states and transitions) so that |L(F3)| = |L(F1)| + 1? |L(R1)| = 5 but your DFA accepts 1000 random strings — is the DFA wrong? (4 + 4 + 4 + 3)

✓ Solution

(g) Idea: only the parity of the current run of 0s matters, and only when the run ends. N = not inside a run (last symbol 1 or start), O = inside a run of odd length, E = inside a run of even length ≥ 2. A 1 or the end of input closes the run: from E that is fatal (1 → D; E is non-accepting).

```
     0   1
→*N  O   N
 *O  E   N
  E  O   D
  D  D   D
(→ start state, * accepting state)
```

0001100: N→O→E→O→N→N→O→E end in E → reject ✓. 01110001: …→O→N … runs 1 and 3 → ends N accept ✓.

(h) State (k, c): k = number of completed a-runs of length exactly 3 (0, 1, 2), c = length of the current a-run capped at 4 (4 means ≥ 4). On a: c → min(c + 1, 4). On b: if c = 3 then k → k + 1 (k = 3 → dead), c

→ 0. At the end the current run also counts, so (k, c) is accepting iff k + [c = 3] ≤ 2. 15 states + dead = 16; a minimisation program confirms 16 is minimal.

```
       a    b
→*0,0  0,1  0,0
 *0,1  0,2  0,0
 *0,2  0,3  0,0
 *0,3  0,4  1,0
 *0,4  0,4  0,0
 *1,0  1,1  1,0
 *1,1  1,2  1,0
 *1,2  1,3  1,0
 *1,3  1,4  2,0
 *1,4  1,4  1,0
 *2,0  2,1  2,0
 *2,1  2,2  2,0
 *2,2  2,3  2,0
  2,3  2,4  D
 *2,4  2,4  2,0
  D    D    D
(→ start state, * accepting state, state 'k,c'; e.g. aaabaaabaaa ends in (2,3) → 3 runs → reject)
```

Note: a single a or a length-2 run never counts; aaaa is a run of length 4, not 3.

(i) Reading the five diagrams (states named by position; missing arrows go to the sink):

FA	Behaviour	Language
i	a moves start → accepting top; from top b returns, a goes to a state that only leads to the sink	(ab)*a — alternating, starting and ending with a
ii	upper branch a^+ then b to an accepting state; lower branch a single b; any further symbol → sink	a*b = {b, ab, aab, …}
iii	a = +1, b = −1 on a counter 0–2; start is the only accepting state; leaving 0…2 → sink	balanced strings (a = '(' , b = ')') with nesting depth ≤ 2, e.g. ε, ab, aabb, abaabb
iv	from the centre (start, accepting) ba or ab returns to the centre; aa/bb patterns → sink	(ab ∪ ba)*
v	alternation until the first aa or bb; after aa a later b accepts, after bb a later a accepts; accept state loops	strings containing aab or bba

Each description was checked by running the drawn automaton on all strings up to length 9.

(j) q1/q3 mean 'last symbol 0' (q3: last two 00); q2/q4 mean 'last symbol 1' (q4: last two 11). L = strings of length ≥ 2 whose last two symbols are equal = (0 ∪ 1)*(00 ∪ 11).

(k) L(F1) = {0^n1 | n ≥ 1} ∪ {10} = 0^+1 ∪ {10}. With δ(d, 1) = d: L(F2) = 0^+1 ∪ 1^+0. |L(F1)| is countably infinite ($\aleph_0$), so |L(F1)| + 1 = |L(F1)|: the 'increase by exactly 1' cannot be measured by cardinality. As sets it is possible to add exactly one string without changing the numbers of states or transitions — e.g. make d accepting: L(F3) = L(F1) ∪ {1}. Last part: yes, the brother is right. L(R1) has only 5 strings, but the DFA accepted 1000 different strings, so L(D1) ⊋ L(R1) and D1 is not equivalent to R1.

Lesson 5 Extra — State Modelling: the Planet Xustaeter (2016-17 Q1)

Stones of three colours R, G, B. When two stones of different colours touch, both are destroyed and two stones of the third colour appear (e.g. R-event: G + B → 2R). The planet fails when all stones have one colour. State = (r, g, b) with r + g + b = N. Classes: **failed** (N,0,0)…; **must-fail**: every path reaches failure; **can't-fail**: no path reaches failure; **might-fail**: some paths do, some don't.

MEMORIZE Invariant: an event changes each difference r − g, g − b by 0 or ±3, so the differences mod 3 never change. Failure (N, 0, 0) has two equal counts mod 3. Hence a state might reach failure only if two of its counts are congruent mod 3.

```
N = 3:  (r,g,b)    class        successors (R-, G-, B-events)
  (0,0,3)    failed       —
  (0,1,2)    can't-fail   (2,0,1)
  (0,2,1)    can't-fail   (2,1,0)
  (0,3,0)    failed       —
  (1,0,2)    can't-fail   (0,2,1)
  (1,1,1)    must-fail    (3,0,0) (0,3,0) (0,0,3)
  (1,2,0)    can't-fail   (0,1,2)
  (2,0,1)    can't-fail   (1,2,0)
  (2,1,0)    can't-fail   (1,0,2)
  (3,0,0)    failed       —
```

```
N = 4:  (r,g,b)    class        successors (R-, G-, B-events)
  (0,0,4)    failed       —
  (0,1,3)    might-fail   (2,0,2)
  (0,2,2)    might-fail   (2,1,1)
  (0,3,1)    might-fail   (2,2,0)
  (0,4,0)    failed       —
  (1,0,3)    might-fail   (0,2,2)
  (1,1,2)    might-fail   (3,0,1) (0,3,1) (0,0,4)
  (1,2,1)    might-fail   (3,1,0) (0,4,0) (0,1,3)
  (1,3,0)    might-fail   (0,2,2)
  (2,0,2)    might-fail   (1,2,1)
  (2,1,1)    might-fail   (4,0,0) (1,3,0) (1,0,3)
  (2,2,0)    might-fail   (1,1,2)
  (3,0,1)    might-fail   (2,2,0)
  (3,1,0)    might-fail   (2,0,2)
  (4,0,0)    failed       —
```

(c) Symmetry: renaming colours does not change behaviour, so states that are permutations of each other behave alike. Reduced scheme: one node per multiset {x, y, z} (sorted counts), with an edge {x,y,z} → {x',y',z'} if some event maps a representative to a representative.

```
N = 6, reduced {x≤y≤z}  class        successors
  (0,0,6)    failed       —
  (0,1,5)    can't-fail   (0,2,4)
  (0,2,4)    can't-fail   (1,2,3)
  (0,3,3)    might-fail   (2,2,2)
  (1,1,4)    might-fail   (0,0,6) (0,3,3)
  (1,2,3)    can't-fail   (0,1,5) (0,2,4) (1,2,3)
  (2,2,2)    might-fail   (1,1,4)
```

For N = 6: must-fail none (besides the failed states); might-fail {0,3,3}, {1,1,4}, {2,2,2}; can't-fail {0,1,5}, {0,2,4}, {1,2,3}. (Computed by search; consistent with the mod-3 invariant.)

Quick Notes — DFA

Revision points for Lessons 3–5.

L3 · DFA definition

- M = (Q, Σ, δ, q0, F), δ : Q × Σ → Q total; exactly one move per (state, symbol).
- Accept iff δ̂(q0, w) ∈ F; accepts ε iff q0 ∈ F. Dead state keeps δ total.
- State = compressed summary of the past; finitely many → finite memory.
- Table = |Q| rows × |Σ| columns; accepting states are only marked.

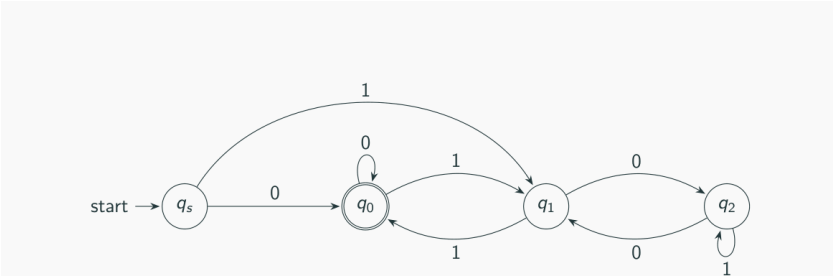
L4 · Design

- Recipe: what to remember → one state per situation → update rule → accepting states → test.
- Contains pattern: states = longest suffix that is a prefix of the pattern; 'does not contain' = flip.
- Remainder DFA: r → (b·r + d) mod m; separate start state if ε must be rejected.
- Binary mod 4: depends only on the last two bits (3 states after minimising: qs ≡ r2, r1 ≡ r3).
- begins with x: wrong first symbol → dead. awa: track first symbol and last symbol.

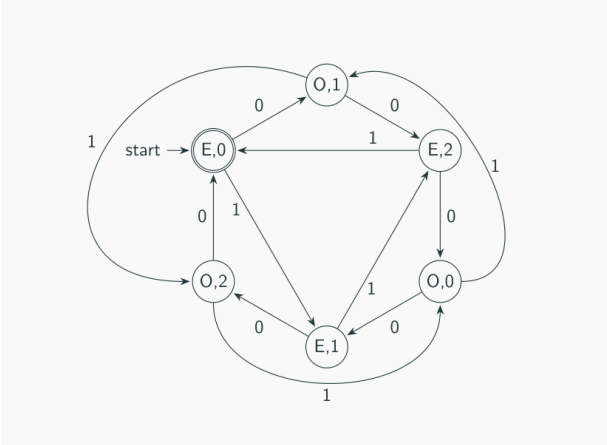
L5 · Advanced

- Product: Q1 × Q2, AND → F1 × F2, OR → either coordinate accepting.
- k-th from end: 2^k states for a DFA (sliding window).
- Sum with reset R: keep sum mod m, R → r0; 'actual sums' → infinitely many states.
- Two-state 'zero / non-zero' fails: remainders 1 and 2 are distinguishable (suffix 2).
- Runs: (completed qualifying runs, current run length capped).
- Reading a DFA: dead states, meaning of each state, shortest accepted strings, loops.

Key Diagrams to Reproduce



Binary multiples of 3 (sl. 27) — same as Fig. 4.2



Product construction (sl. 32) — same as Fig. 5.1